

Part C Why software architecture is important

Software architecture is important because it helps create well organized systems that are easy to understand. Martin Fowler explains architecture as being about the "important stuff" of a software system.

As a software project grows, adding new modules can become difficult. If the code is not well structured, developers will spend a lot of time making small changes. This carries the risk of wasting development cost and time.

Good software architecture helps prevent this issue. When software has solid components and a clear structure, developers can easily find where changes need to be made. They can add new features without modifying many other parts of the system. Developers must perform updates to keep the code clean and healthy.

Software architecture is of great significance for a company. While good internal quality may not be easily visible to customers, it becomes crucial over time. Good architecture should allow new features to be added faster and make updates easier to execute.

Part D What the difference is between software architecture and software design

Software architecture and software design are connected, but they are different. Software architecture is mostly about the important parts of a system and how the whole system works.

Software design is about how we build the software. This includes decisions about classes, functions, parts, and how different pieces of code work together.

Architecture decides the main structure of the program and how the main parts connect. Design then shows how each smaller part is made. I think of software architecture as the big picture of the system. Software design shows the smaller details. So, architecture is about the important things, and design is about how we build and organize the software.

Part E What makes software architecture so difficult.

Software architecture is the big picture of a system. You need to understand what the system needs first, so it takes a lot of time and is not easy. A system can have many parts, and developers must choose which parts really matter.

Another problem is that early choices are hard to change later. At the start of a project, developers must make big choices, but they may not have enough information yet. As the project grows, those early choices might not look so good anymore.

Software changes all the time. Customers ask for new features, and needs can change. Good developers make it easy to handle these changes later. When software gets too complicated, it is hard to understand and fix. Developers need to use the same code style and structure. If everyone writes code in their own way without rules, it makes building the system much harder.

I think the hardest part of software architecture is deciding what is important and what should be easy to change. We cannot make everything flexible, because that makes the system even more complicated. We must choose wisely where we really need flexibility.